Layer-wise Verb Semantics in L2 Writing (BERT + vMF)

Code accompanying the study on how the semantic dispersion of basic verbs changes across CEFR levels in learner English, measured layer by layer in BERT.

For each target verb, the pipeline extracts the verb-token vector from all 13 BERT layers, then quantifies how "spread out" those vectors are at each CEFR level using the von Mises–Fisher concentration parameter (kappa) and mean pairwise cosine distance. Lower kappa (or larger distance) = more dispersed = the verb is being used in a wider range of meanings.

Contents

File	Purpose
`shared_utils.py`	Shared helpers: data loading, verb-context extraction (spaCy), all-layer BERT vector extraction, and vMF kappa estimation. Imported by every phase; not run directly.
`phase2_probing.py`	Layer-wise probing. Finds which layer best predicts CEFR level from the verb vector alone (the "peak layer").
`phase3_vmf.py`	vMF kappa across layers and levels: a kappa heatmap, a best-layer line plot, and a peak-vs-unified layer comparison.
`phase3b_bootstrap.py`	Bootstrap 95% confidence intervals for kappa, to test whether the A1→B1+ decrease is significant.
`phase4b_dispersion_lines.py`	Layer-wise dispersion line plots (kappa and pairwise distance) and the A1−B1+ gap curve, with no dimensionality reduction.
`phase4c_logratio_layers.py`	Reproduces **Fig. 4**: raw κ difference vs. scale-free log-ratio `ln(κ_A1/κ_B1+)` across all layers for the three verbs. Reads `kappa_all_layers.csv` (from phase3); no BERT run needed.
`phase4d_metric_robustness.py`	Reproduces **Fig. 5** (§5.6): vMF κ vs. mean pairwise cosine distance scatter, per verb, with the Pearson correlation r in the legend. Reads `dispersion_all.csv` (from phase4b); no BERT run needed.
`phase5_logratio.py`	Supplementary analysis, kept in the package (not the Fig. 4 code): the log-ratio between CEFR levels at each verb's anchor layer with essay-level bootstrap CIs, to test whether a pair's log-ratio excludes 0.
`JEFLL_ten_verbs.csv`	Input data (see format below).

The phases share one design choice: analysis is done per essay (via the `file` column) so that sentences from the same essay never leak across train/test folds or across bootstrap resamples.

Requirements

- Python 3.9 or later
- Packages:

```
torch
transformers
spacy
scikit-learn
numpy
pandas
matplotlib
```

Installation

```bash
# 1. (recommended) create a virtual environment
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

# 2. install the packages
pip install torch transformers spacy scikit-learn numpy pandas matplotlib

# 3. download the spaCy English model (required)
python -m spacy download en_core_web_sm
```

Notes:

- On first run, `bert-base-uncased` (~440 MB) is downloaded automatically from the Hugging Face Hub, so an internet connection is needed the first time. If you plan to run offline (e.g. at a venue), run any phase once beforehand to cache the model.
- A GPU is optional. The code uses CUDA automatically if available and falls back to CPU otherwise; on CPU the runs are slower but complete.

Input data format

`JEFLL_ten_verbs.csv` (UTF-8) with these columns:

column	description
`file`	essay ID (used for essay-level grouping)
`cefr`	CEFR level: `A1`, `A2`, `B1`, `B2`
`year`	(metadata, not used by the analysis)
`topic`	(metadata, not used by the analysis)
`sentence`	the learner sentence containing the verb
`verb`	the target verb (lemma) for this row

`B1` and `B2` are merged into a single `B1+` level inside `shared_utils.py`, giving the three analysed levels `A1`, `A2`, `B1+`. The bundled data covers ten verbs (`bring, come, get, give, go, like, make, see, take, want`); the default target set in each script is `take make like`.

How to run

Run from the folder that contains the scripts and the CSV. Every script writes to an `output/` folder (created automatically).

Run order: `phase2` → `phase3` → `phase3b` → `phase4b` → `phase4c` → `phase4d` → `phase5`. Each phase depends on outputs of earlier ones — `phase4c` reads the kappa table (`output/kappa_all_layers.csv`) from `phase3`, `phase4d` reads the dispersion table (`output/dispersion_all.csv`) from `phase4b`, and `phase3b` / `phase5` use the anchor layers reported by `phase2`. `phase5` is a retained supplementary analysis and is not required to reproduce the figures.

1. Probing — find the peak layer (`phase2`)

```bash
# all three verbs pooled
python phase2_probing.py --csv JEFLL_ten_verbs.csv --verbs take make like

# each verb probed separately (recommended)
python phase2_probing.py --csv JEFLL_ten_verbs.csv --verbs take make like --per
```

Outputs: `output/probing_curve.png` + `output/probing_results.csv` (pooled), or `output/probing_per_verb.png` + `output/probing_per_verb.csv` (`--per_verb`). The printed per-verb peak layers feed the next phases.

2. vMF kappa (`phase3`)

```bash
# heatmap + line plot at a single layer
python phase3_vmf.py --csv JEFLL_ten_verbs.csv --verbs take make like --best_la

# compare "per-verb peak layer" vs "one unified layer"
python phase3_vmf.py --csv JEFLL_ten_verbs.csv --verbs take make like \
    --compare --peak_layers take=6 make=11 like=11 --unified_layer 6
```

Outputs: `output/kappa_all_layers.csv`, `output/kappa_heatmap.png`, and (depending on flags) `output/kappa_best_layer.png` or `output/kappa_strategy_comparison.png`.

3. Bootstrap confidence intervals (`phase3b`)

```bash
```

```
python phase3b_bootstrap.py --csv JEFLL_ten_verbs.csv --verbs take make like \
    --layers take=6 make=11 like=11 --n_boot 2000 --cluster
```

`--cluster` uses essay-level (more conservative) resampling; omit it for sentence-level. Outputs: `output/kappa_with_ci.csv`, `output/kappa_with_ci.png`.

4. Dispersion line plots (`phase4b`)

bash

```
python phase4b_dispersion_lines.py --csv JEFLL_ten_verbs.csv --verbs take make
    --peak_layers take=6 make=11 like=11
```

Outputs, per verb: `output/dispersion_<verb>.png` and `output/gap_<verb>.png`, plus a combined `output/dispersion_all.csv`.

4c. Fig. 4 — raw difference vs. log-ratio across layers (`phase4c`)

bash

```
python phase4c_logratio_layers.py --kappa_csv output/kappa_all_layers.csv \
    --verbs take make like
```

Reproduces the paper's Fig. 4 directly from the kappa table written by phase3. Panel (a) plots `κ(A1) − κ(B1+)` per layer; panel (b) plots the scale-free `ln(κ(A1)/κ(B1+))`. The contrast is the point: the raw difference is inflated at the shallow (anisotropic) layers, while the log-ratio stays bounded and relocates `make`'s peak to L11. Output: `output/fig_raw_vs_logratio.png`.

4d. Fig. 5 — metric robustness (`phase4d`)

bash

```
python phase4d_metric_robustness.py --disp_csv output/dispersion_all.csv \
    --verbs take make like
```

Reproduces Fig. 5 (§5.6) from the dispersion table written by phase4b. It scatters, per verb, every (layer, level) point as vMF κ (x) against mean pairwise cosine distance (y), with the per-verb Pearson r in the legend. The strong negative correlation shows the two dispersion measures agree, so the conclusions do not depend on which metric is used. Output: `output/fig_metric_robustness.png`.

5. Log-ratio significance (supplementary, `phase5`)

bash

```
python phase5_logratio.py --csv JEFLL_ten_verbs.csv --verbs take make like \
    --layers take=6 make=11 like=11 --n_boot 2000 --cluster
```

A supplementary analysis kept in the package (it does not feed Fig. 4). It adds an essay-level bootstrap CI to the between-level log-ratio at each verb's anchor layer (pairs `A1→A2`, `A2→B1+`, `A1→B1+`); a CI that excludes 0 is treated as reliable. This provides a significance test for the log-ratio to sit alongside the κ CIs from phase3b. Outputs: `output/logratio_ci.csv`, `output/logratio_ci.png`.

Note: `like` has two probing peaks (L2 and L11). Run it once per anchor to check both, e.g. `--layers like=2` and `--layers like=11`.

Common options

- `--csv`: path to the input CSV (default `JEFLL_ten_verbs.csv`).
- `--verbs`: target verbs (default `take make like`).
- `--model`: Hugging Face model name (default `bert-base-uncased`).
- Peak layers are passed as `verb=layer`, e.g. `take=6 make=11 like=11`.

Reproducibility

Random operations (subsampling, CV splits, bootstrap) use fixed seeds (`random_state=42`), so repeated runs give the same numbers.